EXPERIMENT-6

NAME: Harsan.J ROLL NO: 240701179

1. COMMAND LINE INTERFACE:

PYTHON CODE:

```
import psutil
import time
import os
import sys

def clear_screen():
    if os.name == "nt":
        os.system("cls")
    else:
        os.system("clear")

def get_stats():
    cpu = psutil.cpu_percent(interval=1)
    ram = psutil.virtual_memory().percent
    disk = psutil.disk_usage('/').percent

    battery = psutil.sensors_battery()
    if battery:
        status = "Charging" if
battery.power_plugged else "Not Charging"
        battery_text = f"{battery.percent} %
({status})"
    else:
        battery_text = "Not Available"

    return cpu, ram, disk, battery_text

try:
    while True:
        clear_screen()
        cpu, ram, disk, battery = get_stats()

        print("=" * 30)
        print("  SYSTEM PERFORMANCE (CLI)")
        print("=" * 30)
        print(f"CPU Usage     : {cpu} %")
        print(f"RAM Usage     : {ram} %")
        print(f"Disk Usage    : {disk} %")
        print(f"Battery Level : {battery}")
        print("\nPress Ctrl+C to exit")

        time.sleep(1)
```

```
except KeyboardInterrupt:
    clear_screen()
    print("CLI System Monitor exited.")
    sys.exit()
```

OUTPUT:

2. GRAPHICAL USER INTERFACE:

PYTHON CODE:

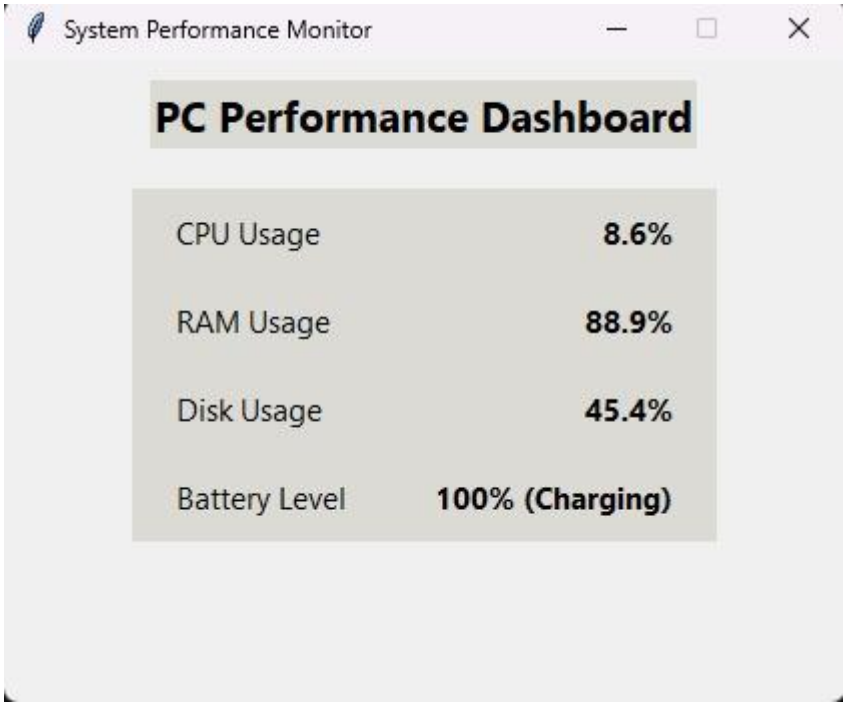
```
import tkinter
as tk

from tkinter
import ttk

import psutil


# -------------------
GUI SETUP ------
-------------

root = tk.Tk()

root.title("Syste
m Performance
Monitor")

root.geometry(
"420x320")

root.resizable(F
alse, False)


style =
ttk.Style()

style.theme_us
e("clam")


# -------------------
FUNCTIONS -----
--------------

def
update_stats():

    # CPU

    cpu =
psutil.cpu_perc
ent()


cpu_label.confi
g(text=f"{cpu}%
")
```

```
    # RAM
    ram = psutil.virtual_memory().percent
    ram_label.config(text=f"{ram}%")

    # Disk
    disk = psutil.disk_usage('/').percent
    disk_label.config(text=f"{disk}%")

    # Battery
    battery = psutil.sensors_battery()
    if battery:
        status = "Charging" if battery.power_plugged else "Not Charging"
        battery_label.config(text=f"{battery.percent}% ({status})")
    else:
        battery_label.config(text="Not Available")
```

```
root.after(1000,
update_stats)  #
update every
second



# -------------------
UI LAYOUT ------
-------------

title =
ttk.Label(root,
text="PC
Performance
Dashboard",
font=("Segoe
UI", 16, "bold"))

title.pack(pady=
10)


frame =
ttk.Frame(root)

frame.pack(pad
y=10)


def
stat_row(name,
row):

    label =
ttk.Label(frame,
text=name,
font=("Segoe
UI", 11))

label.grid(row=r
ow, column=0,
padx=20,
pady=10,
sticky="w")


    value =
ttk.Label(frame,
text="0%",
```

```
font=("Segoe
UI", 11, "bold"))


value.grid(row=
row, column=1,
padx=20,
pady=10,
sticky="e")

    return value



cpu_label =
stat_row("CPU
Usage", 0)

ram_label =
stat_row("RAM
Usage", 1)

disk_label =
stat_row("Disk
Usage", 2)

battery_label =
stat_row("Batte
ry Level", 3)


# -------------------
START ------------
-------

update_stats()

root.mainloop()
```

OUTPUT:



3. VOICE USER INTERFACE:

PYTHON CODE:

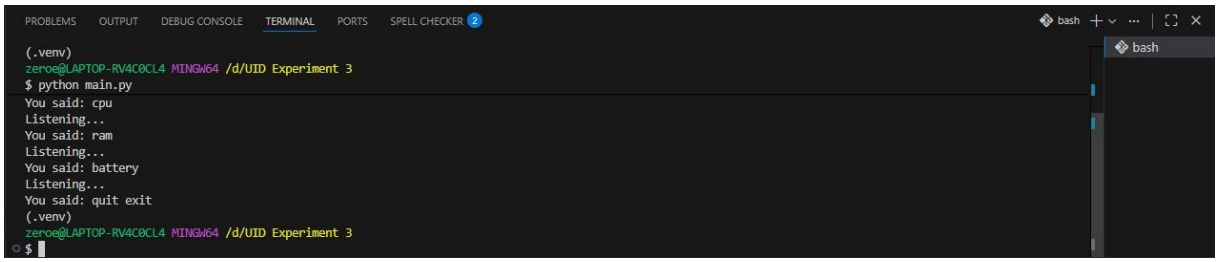
```
import psutil

import pyttsx3

import speech_recognition

    as sr


engine = pyttsx3.init()


def speak(text):

    engine.say(text)

    engine.runAndWait()


def listen():

    r = sr.Recognizer()

    with sr.Microphone() as

    source:

        print("Listening...")


    r.adjust_for_ambient_noi

    se(source)

        audio = r.listen(source)
```

```
    try:
        command =
    r.recognize_google(audio)
    .lower()
        print("You said:",
    command)
        return command
    except:
        return ""

def cpu_usage():
    usage =
    psutil.cpu_percent(interv
    al=1)
    speak(f"CPU usage is
    {usage} percent")

def ram_usage():
    ram =
    psutil.virtual_memory().p
    ercent
    speak(f"RAM usage is
    {ram} percent")

def disk_usage():
    disk =
    psutil.disk_usage('/').perc
    ent
    speak(f"Disk usage is
    {disk} percent")

def battery_status():
    battery =
```

```
    psutil.sensors_battery()
    if battery:
        status = "charging" if
    battery.power_plugged
    else "not charging"
        speak(f"Battery level is
    {battery.percent} percent
    and it is {status}")
    else:
        speak("Battery
    information is not
    available")

speak("Voice system
    monitor activated")

while True:
    command = listen()

    if "cpu" in command:
        cpu_usage()

    elif any(w in command for
    w in ["ram", "memory",
    "mem"]):
        ram_usage()

    elif "disk" in command or
    "storage" in command:
        disk_usage()

    elif "battery" in command
    or "power" in command:
        battery_status()
```

```
    elif "exit" in command or
    "quit" in command:
        speak("Shutting down
    system monitor")
        break
```

OUTPUT :



4. USER SATISFACTION COMPARISON:

PYTHON CODE:

```
def survey():
    print("Rate your satisfaction with the following interfaces (1-5):")

    # Get user input for each interface
    try:
        cli_satisfaction = int(input("CLI (1-5): "))
        gui_satisfaction = int(input("GUI (1-5): "))
        vui_satisfaction = int(input("VUI (1-5): "))

        # Ensure valid ratings
        if not (1 <= cli_satisfaction <= 5 and 1 <= gui_satisfaction <= 5 and 1 <= vui_satisfaction <= 5):
            print("Please enter ratings between 1 and 5 only.")
            return

        # Display the ratings
        print("\nYour satisfaction ratings:")
        print(f"CLI: {cli_satisfaction}")
```

```
print(f"GUI: {gui_satisfaction}")
```

```
        print(f"VUI: {vui_satisfaction}")


        # Calculate the average satisfaction
        avg_satisfaction = (cli_satisfaction + gui_satisfaction + vui_satisfaction) / 3
        print(f"\nAverage Satisfaction Score: {avg_satisfaction:.2f}")


    except ValueError:
        print("Invalid input! Please enter numbers between 1 and 5.")


# Run the survey function
if __name___== "__main__":
    survey()
```

```
D:\230701304>python survey.py
Rate your satisfaction with the following interfaces (1-5):
CLI (1-5): 4
GUI (1-5): 5
VUI (1-5): 3

Your satisfaction ratings:
CLI: 4
GUI: 5
VUI: 3

Average Satisfaction Score: 4.00

D:\230701304>
```